Assignment 1. Build an NLP System

Objective

Build a small NLP system with an AI coding assistant (use it freely, but think through the process how you manage it? What do you put into its context? What do you put into agents file or convention file to control code quality? Be ready to explain how the system works. Identify the purpose of each component. Evaluate the quality of the output (if you have multiple components, quality evaluation must be present at each stage).

Task

Choose an app idea based on NLP components. Build a working prototype that accepts input and produces useful output. Select the tools, model, and interface. You can use a simple web interface. Deployment is optional but the app must be possible to run, for example one of the deliverables will be video recording of the system demo, see more details in “Submission” section.
You can use retrieval-augmented generation (RAG), agents, data cleaning, classification, information extraction, search, summarization and other types of tasks or components.
In development, follow CRISP-DM cycle, it should be reflected in your report how you applied it to the task.
Create SKILLS for your agentic development flow, so you can make the task faster and better.

Project Examples

- Build a **Recruiter assistant** which helps finding best candidates. It must be able to collect requirements from the user about the job posting, then search through a preexisting database of candidates (find data online or populate with fake one, open datasets with CVs also exists). When requirements are collected, the assistant must start searching for the best candidate based on business logic you see reasonable. Check smth like this source to get insights on what you can use in such a system: https://www.klearskill.com/resources/blog/interview-evaluation-forms-templates. When the best candidates have been selected, the system must create a personal message for those candidates which a Recruiter would use to pursue them on the linkedin (think grounding the message based on the info in candidate CV). Make sure you evaluate each component of the system and can report metrics.

- Build an **HR assistant** that answers employee questions about HR procedures. Example topics include leave, onboarding, benefits, and internal requests. Use a small collection of company policies. You can create fictional policies for this task. The assistant must

collect the information needed to answer each question. It must retrieve the relevant policy and identify the source in its answer. It must also have a function where an employee requests a ‘leave’ (any type, you can define) and the system must track it (approve/decline), use a database to handle that and access it via LLM only. If the policies do not support an answer, it must route to HR. Evaluate each component. Report metrics for information collection, retrieval success, answer correctness, source support, and correct referral to HR.

- Build a **business analysis assistant** that helps users prepare specifications and divide work into tasks. The assistant must collect the system goals, users, requirements, constraints, and team roles. It must identify missing or conflicting requirements. It must ask follow-up questions before it produces the final documents. The assistant must produce a specification, a task breakdown, acceptance criteria, and a RACI matrix. Each task must link to a requirement. Each RACI activity must have one accountable role. The assistant must distinguish confirmed requirements from proposed assumptions. Evaluate each component. You may use multiple agents as a core of this system where each agent has own narrowed professional skill.

- Build a **company stock price awareness assistant.** A system which could monitor trends, company stock price, news which could influence that price, you can add cryptocurrency monitoring as well. It must not only visualise collected data, but also analyse it and provide suggestions. Potentially you may add a ‘stock buying simulator’ where your agent will also ‘buy’ stocks when the other agent’s suggestion is highly confident so we can see over time if your agent is capable of making a profit. You can use historical data for the simulation process as well.

- Build a **DevOps assistant** that generates **Terraform configuration** for a small AWS service. The assistant must collect system requirements, including workload, network access, availability, storage, and cost constraints. It must retrieve relevant AWS best practices documentation. It must then generate Terraform configuration and explain the main infrastructure choices. The generated configuration must be a valid Terraform config. Use Terraform formatting and validation tools. No need for real running of the terraform config, unless you have resources for that and are curious about the results. For the task it is enough to end up with valid structure.

You can choose another task but the examples above can give you a hint on what we expect. If you select the same task as another teammate, we will check if you don’t have the same implementation, the copying is strictly forbidden.
YOU CANNOT build vanilla RAG and call it a day, such a result will not be graded.

Procedure

1. Define the Task (you can extend business requirement as long as you understand how to evaluate the system)

2. Prepare the Data (check existing datasets before generating synthetic)

3. Describe the Components

Draw a diagram that shows the data flow. Describe the purpose, input, and output of each component. Identify model calls and external tool calls. Explain how one input passes through the complete system. You can use LLM for this task, but you must control the process.

4. Build the Prototype

Give the AI coding assistant a clear request and make sure you manage the instructions to your agent not only through cycles of prompting. You must create agents md file or convention md file or any other way to handle and steer agentic behaviour and context. **Follow Crisp-dm** by instructing your coding agent to do it properly, put instructions into the agents md or similar file.

5. Evaluate the System

Evaluation is required for every component of the project. It can be also done by an LLM but you must be able to distinguish between high and low scores, think of how to improve it and explain why scores are not as good as they could be (if the case). The selection of metrics will also be judged, so make sure you choose proper metrics.

Submission

- **Code and short report:** Submit the code or notebook with setup and run instructions. Github repo is good as well. Make sure it contains also high level system diagrams, description of business logic you decided to keep in the scope and evals results, it may be placed in PDF or a notebook.
- **Video recording:** Be ready to run the system locally and record the explanation of how it works. It must be video max 10 minutes showing your report docs, diagrams, the running system demo:
    - speak about system requirements you selected for the scope of the task,
    - speak about system components (your diagram) and why you selected those,

    - run the app and go through some cases (show how it behaves), you can turn on LLM tracing to log the process
    - show and explain evaluations,
    - show how you set up an agentic coding env, control its context, behaviour, etc.

Assessment Criteria

- **System understanding:** Explain each component and trace a flow of input through the system (walkthrough). **- 4 pts**
- **Working prototype:** Complete the selected task. - **4 pts**
- **Evaluation:** evaluation must be done for each component of the system. **- 4 pts**
- **Use of the AI coding assistant:** make sure you properly manage the coding agent via instructional md files and be ready to briefly reflect on that. **- 3pts**

Useful resources

- Free API LLMs (they use outputs for training): https://openrouter.ai/collections/free-models